

Request Body Stream Consumption Problem — Detailed Notes

("Why does the request body become empty/null after reading it once?")

1. The Core Concept

When an HTTP request arrives at the server, the request body is **not** automatically loaded into memory as a String or object. It exists as a **stream** (`InputStream`) — a one-way pipe of bytes coming directly from the client's TCP connection.

Analogy: A stream is like a **water pipe with a one-time valve** — once you read (drain) the data from it, it's gone. There's no built-in way to "rewind" it back to the start unless something explicitly saved a copy along the way.

The Problem Scenario:

1. A **Filter** reads the request body (e.g., to log it)
2. Then the **Controller** tries to read the same body again (e.g., via `@RequestBody`)
3. The Controller receives an **empty stream** — because the Filter already consumed all the bytes

Technically, it's not Java's `null` — it's an **exhausted/empty stream**, which causes deserialization to fail, return default/empty values, or throw an exception.

2. Why This Happens (Technical Reason)

- `HttpServletRequest.getInputStream()` returns a `ServletInputStream` — backed **directly** by the underlying TCP socket's read buffer.
- Once bytes are read off that buffer, they are **removed/consumed** — similar to reading a file line-by-line.
- The raw **Servlet API provides no built-in caching or replay mechanism**. The container assumes the body will be read **exactly once**, typically inside the Controller during request processing.
- This is a **fundamental limitation of I/O streams in general** (not just Java) — a stream represents a one-directional flow of data, not a stored/reusable data structure.

3. Example That Demonstrates the Problem

Faulty Filter (reads raw stream directly):

```
@Component
public class BadLoggingFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                   FilterChain filterChain) throws ServletException, IOException {

        // Filter reads the body here to log it
        String body = request.getReader().lines().collect(Collectors.joining());
        System.out.println("Request Body: " + body);

        // Passes the SAME original (now-consumed) request down the chain
        filterChain.doFilter(request, response);
    }
}
```

Controller:

```
@RestController
public class UserController {

    @PostMapping("/users")
    public ResponseEntity<String> createUser(@RequestBody UserDto userDto) {
        // userDto will have EMPTY/NULL fields here!
        System.out.println("Received: " + userDto);
        return ResponseEntity.ok("Created");
    }
}
```

What Actually Happens (Step-by-Step):

1. Client sends: `{"name": "John", "age": 25}`
2. Filter reads the raw `InputStream` directly and prints it correctly: Request Body: `{"name": "John", "age": 25}`
3. Request proceeds down `chain.doFilter()` to the Controller
4. Jackson tries to deserialize the `@RequestBody` — but the underlying stream is **already fully drained**
5. **Result:** `userDto` comes back with all null/default field values, OR Spring throws `HttpMessageNotReadableException` ("Required request body is missing") — depending on strictness settings

4. Solution 1 — Using `ContentCachingRequestWrapper`

Key Idea: This wrapper doesn't pre-read the stream itself. It **intercepts** reads made by whatever reads it downstream (like Jackson during `@RequestBody` binding) and **duplicates** the bytes into an internal cache **as they pass through** — without consuming them prematurely.

```
@Component
@Order(1)
public class RequestCachingFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                   FilterChain filterChain) throws ServletException, IOException {

        // Wrap the request — does NOT consume the stream yet
        ContentCachingRequestWrapper wrappedRequest = new ContentCachingRequestWrapper(request);

        // Pass the WRAPPED request down — Controller reads from this wrapper,
        // and bytes get cached internally AS they are read
        filterChain.doFilter(wrappedRequest, response);

        // AFTER the Controller has read the body (via the wrapper),
        // we can now safely access the cached copy for logging
        String body = new String(wrappedRequest.getContentAsByteArray(),
                                wrappedRequest.getCharacterEncoding());

        System.out.println("Request Body: " + body);
    }
}
```

Why This Works:

- The wrapper **passively observes** reads instead of actively consuming the stream itself.
- The Controller still receives the full, intact body — nothing is consumed before it gets there.
- The Filter then reads the **cached copy** (a `byte[]`) — not the live stream — so there's no conflict between Filter and Controller.

Limitation: This only works if the body is **actually read by something** downstream (e.g., a `@RequestBody` parameter). If nothing reads it, the cache stays empty. Also, it doesn't help if you need to inspect the body **before** calling `chain.doFilter()` (e.g., to reject a request based on its content).

5. Solution 2 — Custom Reusable Request Wrapper (Multi-Read Support)

When Needed: If you need to read the body **more than once** at different stages — e.g., once in a security filter (to inspect/validate before proceeding), once in a logging filter, and once in the Controller — `ContentCachingRequestWrapper` alone isn't sufficient. You need a wrapper that fully reads the body **once** into memory and then always returns a **fresh stream copy** on every subsequent read call.

Step 1: Custom `HttpServletRequestWrapper`

```

public class CachedBodyHttpServletRequest extends HttpServletRequestWrapper {

    private final byte[] cachedBody;

    public CachedBodyHttpServletRequest(HttpServletRequest request) throws IOException {
        super(request);
        InputStream requestInputStream = request.getInputStream();
        this.cachedBody = StreamUtils.copyToByteArray(requestInputStream);
    }

    @Override
    public ServletInputStream getInputStream() {
        return new CachedBodyServletInputStream(cachedBody);
    }

    @Override
    public BufferedReader getReader() {
        ByteArrayInputStream byteArrayInputStream = new ByteArrayInputStream(cachedBody);
        return new BufferedReader(new InputStreamReader(byteArrayInputStream));
    }
}

```

Step 2: Custom `ServletInputStream`

```

public class CachedBodyServletInputStream extends ServletInputStream {

    private final ByteArrayInputStream buffer;

    public CachedBodyServletInputStream(byte[] contents) {
        this.buffer = new ByteArrayInputStream(contents);
    }

    @Override
    public int read() {
        return buffer.read();
    }

    @Override
    public boolean isFinished() {
        return buffer.available() == 0;
    }

    @Override
    public boolean isReady() {
        return true;
    }

    @Override
    public void setReadListener(ReadListener readListener) {
        // not needed for typical synchronous use
    }
}

```

Step 3: Usage in a Filter

```

@Component
public class BodyInspectionFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                   FilterChain filterChain) throws ServletException, IOException {

        CachedBodyHttpServletRequest cachedRequest = new CachedBodyHttpServletRequest(request);

        // Read the body here BEFORE proceeding – e.g., for validation/inspection
        String body = new String(cachedRequest.getInputStream().readAllBytes());
        System.out.println("Body (pre-check): " + body);

        // The Controller downstream can ALSO read it fully,
        // since getInputStream() always returns a FRESH stream from the cached byte array
        filterChain.doFilter(cachedRequest, response);
    }
}

```

Why This Works:

- The full body is read **once** into a `byte[]` inside the wrapper's constructor.
- Every subsequent call to `getInputStream()` or `getReader()` returns a **brand new stream** built from that same stored `byte[]` – so it can be read **unlimited times**, by any number of filters and finally the Controller.

6. Comparison of the Two Solutions

Aspect	ContentCachingRequestWrapper	Custom CachedBodyHttpServletRequest
Who reads first	Downstream code (Controller/Jackson) – wrapper just observes	The wrapper itself reads and stores the body immediately in constructor
Can inspect body BEFORE <code>chain.doFilter()</code> ?	No – cache is empty until something downstream reads it	Yes – full body is available immediately after wrapping
Reusable multiple times?	Only once cached (after being read once downstream)	Yes – unlimited reads, since a fresh stream is generated each time
Complexity	Simple – built into Spring, ready to use	Requires writing custom wrapper + input stream classes
Best for	Logging response/request bodies for observability	Validating/inspecting the body early (e.g., security checks, signature verification) before passing it on

7. Golden Rules to Remember

1. **Always pass the wrapped request object down the filter chain** (`chain.doFilter(wrappedRequest, response)`) – never the original – or nothing gets cached/reused correctly.
2. **Never read the raw stream directly in a Filter** if the Controller also needs to read it – this will exhaust the stream and cause the Controller to receive an empty body.
3. With `ContentCachingRequestWrapper`, always read `getContentAsByteArray()` **after** `chain.doFilter()` completes – not before – since the cache fills up progressively as the body is read downstream.
4. If you need to inspect the body **before** forwarding the request (e.g., for validation, signature checks, security filters), use a **custom reusable wrapper** (Solution 2) instead.
5. This entire problem applies **only to the request body** – response bodies have an analogous (but reversed) problem solved by `ContentCachingResponseWrapper`, which requires calling `copyBodyToResponse()` to actually send the buffered data to the client.

8. Quick Summary (for revision)

- **Root cause:** The HTTP request body is a **one-time-readable `InputStream`**, backed directly by the raw socket connection – not a reusable/stored data structure.
- **Symptom:** If a Filter reads the raw request body first, the Controller downstream receives an **empty/exhausted stream**, resulting in null/default fields or a `HttpMessageNotReadableException`.

- **Fix (simple, single extra read):** Use `ContentCachingRequestWrapper` — pass the wrapped request through `chain.doFilter()`, then read `getContentAsByteArray()` afterward.
- **Fix (advanced, multiple reads / early inspection):** Build a custom `HttpServletRequestWrapper` that reads the entire body **once** into a `byte[]` in its constructor, and returns a **fresh stream** from that array on every `getInputStream()` / `getReader()` call — allowing unlimited re-reads.
- **Golden rule:** Always forward the **wrapped** request (not the original) through the filter chain, or the caching mechanism won't work.